

OS161

COME VIENE CONFIGURATO UN KERNEL

Quando creiamo una nuova configurazione del kernel in **OS/161**, non tutti i file sorgente devono essere compilati sempre. Alcune funzionalità (ad esempio una nuova feature chiamata **project**) devono essere presenti solo in determinate configurazioni del kernel.

Per questo motivo OS/161 utilizza un sistema di **opzioni**, che permette di decidere quali parti del codice compilare e quali ignorare.

1) Dichiarare una nuova opzione

Diciamo al sistema di configurazione che esiste una nuova opzione.

Apriamo il file : **kern/conf/conf.kern** e aggiungiamo :

defoption project

optfile project syscall/project.c (oppure file **syscall/project.c** option **project**)

defoption project → dice al sistema di configurazione che esiste un'opzione chiamata **project** (la DICHIARA, NON la ABILITA).

Optfile... → OS161 non cerca automaticamente tutti i file .c presenti nel progetto, ma compila solo quelli dichiarati nel sistema di configurazione. Attraverso questa associazione diciamo che il file **syscall/project.c** deve essere compilato solo se l'opzione **project** è attivata!

2) Creare una nuova configurazione del kernel

Ogni configurazione del kernel ha un suo file nella cartella **kern/conf**. Supponiamo di voler creare una configurazione chiamata **PROJECT**.

Si crea il file: **kern/conf/PROJECT** con contenuto:

options project

In questo modo ABILITIAMO / ATTIVIAMO l'opzione **project**

3) Generare la configurazione del kernel

Ci spostiamo nella cartella: **kern/conf** ed eseguiamo **./config PROJECT**

Questo NON compila il kernel ma legge il file **conf.kern** e il file **PROJECT**. Controlla quali opzioni sono abilitate e prepara automaticamente tutto il necessario per la compilazione, genera la cartella : **kern/compile/PROJECT** e crea automaticamente il file **opt-project.h** che contiene:

#define OPT_PROJECT 1 → se in **PROJECT** è presente **options project**

#define OPT_PROJECT 0 → se non è presente.

In questo modo il sistema di configurazione trasforma una opzione del kernel in una macro del linguaggio C.

4) Includere nei file che necessitano di quella opzione: **#include "opt-project.h"**

Possiamo usare :

#include "opt-project.h"

#if OPT_PROJECT

project_init();

#endif

Il processore PRIMA della compilazione legge **OPT_PROJECT**.

Se 1 → mantiene il codice

- Se 0→ elimina il codice (come se non fosse mai esistito)
- 5) COMPILAZIONE: in kern/compile/PROJECT
in kern/compile/PROJECT: **bmake depend** (analizza tutti i file che devono essere compilati, costruisce le dipendenze e decide quali file fanno parte del kernel), **bmake** (compila effettivamente i file), **bmake install** (installa il nuovo kernel nella directory di esecuzione)
- 6) Avvio del kernel
In **os161/root**: **sys161 kernel** per avviarlo

L'esecuzione (in kern/conf) di ./config PROJECT, seguita dal comando bmake depend (eseguito in kern/compile/PROJECT), è sufficiente affinché il file opzionale syscall/project.c sia compilato quando si esegue bmake (in kern/compile/PROJECT). SI

Il file opt-project.h viene generato solo se l'istruzione options project compare nel file PROJECT NO (viene generato sempre, anche senza l'istruzione, che cambia solo il valore associato a OPT_PROJECT)

User process

OS/161 esegue programmi **UTENTE** (user process) sopra il kernel. Il processore opera in due modalità di esecuzione:

- **User Mode** → esegue il codice dei programmi utente con privilegi limitati.
- **Kernel Mode** → esegue il codice del sistema operativo con privilegi completi

Un **processo** rappresenta un contenitore di risorse e comprende: codice del programma; segmento dati; heap; **Address Space** (spazio di indirizzamento virtuale); file aperti; uno o più **thread**.

Un **thread** rappresenta il flusso di esecuzione del processo e possiede: registri della CPU; stato di esecuzione; contesto (t_context); **Kernel Stack**.

Il collegamento tra thread e processo è mantenuto dal campo: **thread->t_proc**, che punta alla struct proc del processo di appartenenza. Ogni thread di uno stesso processo condivide le risorse contenute nella struct proc, in particolare l'**Address Space**, ma mantiene un proprio contesto di esecuzione e un proprio kernel stack.

CREAZIONE DI UN PROCESSO UTENTE

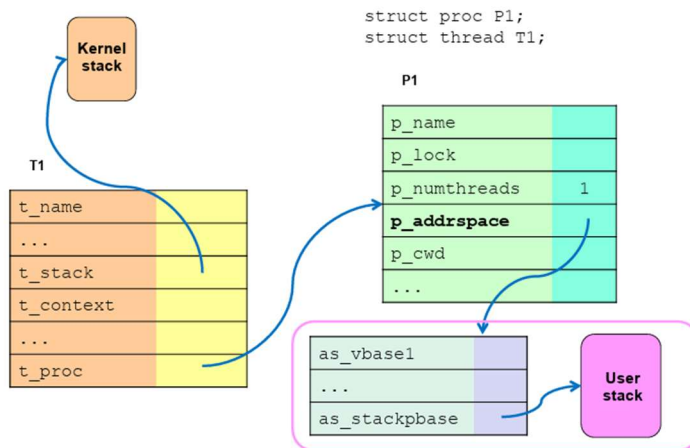
Quando viene eseguito un programma utente (ad esempio ls, cat o testbin/palin), OS/161:

1. crea una nuova **struct proc** (proc_create_runprogram());
2. crea un thread kernel (thread_fork());
3. il thread esegue cmd_progthread();
4. cmd_progthread() invoca runprogram();
5. viene creato l'Address Space;
6. viene caricato il file eseguibile ELF;
7. il thread passa da **Kernel Mode** a **User Mode** iniziando l'esecuzione del programma.

Ogni thread associato ad un processo utente possiede **due stack**:

- **USER STACK**: È utilizzato durante l'esecuzione del programma utente (**User Mode**). Contiene: variabili locali; parametri delle funzioni; indirizzi di ritorno. Appartiene all'**Address Space** del processo ed è quindi una risorsa del processo.
- **KERNEL STACK**: utilizzato quando il thread entra nel kernel (**Kernel Mode**) a causa di: system call; interrupt; eccezioni (trap). Contiene: variabili locali delle funzioni kernel; trapframe; dati temporanei del kernel. E' una risorsa privata del thread.

Process Stack(s)



ADDRESS SPACE: rappresenta la memoria virtuale del processo. Contiene: segmento codice; segmento dati; heap; User Stack.

Nella struct proc abbiamo: **struct addrspace *p_addrspace;**

Ogni processo possiede un Address Space indipendente, condiviso da tutti i thread appartenenti a quel processo.

SYSTEM CALL

Un processo utente **non può accedere direttamente all'hardware o alle strutture del kernel**. Se ha bisogno di un servizio del sistema operativo (ad esempio leggere un file, scrivere sul terminale o creare un processo), deve richiedere l'intervento del kernel tramite una **system call** (meccanismo con cui un processo utente chiede al kernel di eseguire un'operazione privilegiata, tipo read(fd, buf, n). Questa operazione non viene eseguita direttamente dal processo, ma provoca un passaggio da User Mode a Kernel Mode).

Il flusso è il seguente: processo utente chiama system call → trap(eccezione sw) → ingresso nel kernel che esegue la richiesta → ritorno a processo utente.

Quando il processore entra nel kernel a causa di una system call, DEVE SALVARE lo stato del processo che era in esecuzione: **TRAPFRAME**, così da poter riprendere l'esecuzione dall'istruzione in cui era stata interrotta.

TRAPFRAME

Contiene principalmente:

- Program Counter (`tf_epc`);
- Stack Pointer (`tf_sp`);
- registri generali;
- registri degli argomenti (`a0-a3`);
- registro di stato.

Si tratta di una struttura dati che contiene il contesto completo del processore (contesto CPU) quando il programma entra nel kernel. Così il kernel può eseguire la system call e ripristinare successivamente il processo esattamente dal punto in cui era stato interrotto.

I registri principali sono :

<code>v0</code>	numero della system call
<code>a0-a3</code>	argomenti della system call
<code>v0</code>	valore restituito al termine
<code>a3</code>	indicatore successo/errore
<code>epc</code>	indirizzo dell'istruzione utente

Viene allocato nello **STACK DEL KERNEL (mai nello stack utente, che appartiene al processo e non è affidabile in quanto potrebbe contenere indirizzi non validi, essere modificato dal processo...).**
Viene utilizzato durante una SYSTEM CALL.

SWITCHFRAME

Questo a differenza del Trapframe NON viene usato durante una system call, ma durante il CONTEXT SWITCH (quando il kernel sospende un thread e ne esegue un altro). Contiene solo i registri necessari a sospendere un thread e riprenderne un altro, viene usato in `mi_switch()`.

Se un processo utente esegue `read(fd, buf, n)` passando un puntatore non valido, il kernel verifica la validità dell'indirizzo mediante funzioni come **`copyin()`** e **`copyout()`**. Se il puntatore non è valido, la copia fallisce e la system call restituisce un errore senza compromettere l'esecuzione del kernel, solitamente EFAULT.

HEADER DEL KERNEL: **`kern/include/syscall.h`** → contiene I prototipi delle funzioni che implementano le system call.

Dentro:

```
#include "opt-syscalls.h"
```

```
#if OPT_SYSCALLS
```

```
int sys_write(int fd, userptr_t buf, size_t len);
```

```
int sys_read(int fd, userptr_t buf, size_t len);
```

```
void sys__exit(int status);
```

```
#endif
```

(per header interni al progetto usiamo `#include <>` , per gli header esportati dal kernel usiamo `#include <>`)

DISPATCHER DELLE SYSTEM CALL: **`kern/arch/mips/syscall/syscall.c`**

La funzione principale è : **void syscall(struct trapframe *tf)** che deve capire quale system call è stata richiesta, recuperare gli argomenti, chiamare la funzione kernel corretta, scrivere nel trapframe il risultato / errore, far riprendere il programma utente.

ESEMPIO: write(1, buffer,10)

a0-a3 = parametri

```
v0 = numero della system call write
a0 = 1
a1 = indirizzo di buffer
a2 = 10
```

Si esegue syscall, genera una trap, si passa da User Mode a Kernel Mode, prima di entrare nel dispatcher lo stato della CPU viene salvato in un trapframe. LATO KERNEL:

```
void
syscall(struct trapframe *tf)
{
    int callno;
    int32_t retval;
    int err = 0;

    KASSERT(curthread != NULL);
    KASSERT(curthread->t_curspl == 0);
    KASSERT(curthread->t_iplhigh_count == 0);

    callno = tf->tf_v0;
```

```
    retval = 0;

    switch (callno) {
        case SYS_reboot:
            err = sys_reboot(tf->tf_a0);
            break;

        case SYS_time:
            err = sys_time((userptr_t)tf->tf_a0,
                          (userptr_t)tf->tf_a1);
            break;
```

```
        case SYS_write:
            err = sys_write((int) tf->tf_a0, (userptr_t) tf->tf_a1, (size_t) tf->tf_a2);
            break;

        case SYS_read:
            err = sys_read((int) tf->tf_a0, (userptr_t) tf->tf_a1, (size_t) tf->tf_a2);
            break;

        case SYS_exit:
            //abbiamo chiamato la syscall e ora dobbiamo terminare il processo
            sys_exit((int) tf->tf_a0);
            break;
```

SYS_reboot e SYS_time sono già fornite dal kernel

Struct trapframe *tf → tf punta al trapframe salvato nello stack kernel del thread corrente.

Contiene: tf->tf_v0 = numero della system call (callno), tf->tf_a0 – tf->tf_a3 (parametri/argomenti funzione), tf_epc = indirizzo dell'istruzione syscall.

La riga: callno=tf->tf_v0 recupera il numero della system call richiesta.

Il dispatcher usa uno switch sul numero della system call, che associa ogni numero della system call alla funzione kernel corrispondente. In caso di successo abbiamo retval, altrimenti err.

Quando la system call termina, il kernel deve comunicare il risultato al programma utente.

CASO DI SUCCESSO:

tf->tf_v0 = retval; → valore restituito

tf->tf_a3 = 0; → operazione riuscita

CASO DI ERRORE:

tf->tf_v0 = err; → codice errore

tf->tf_a3 = 1; → operazione fallita

In **kern/syscall/file_syscalls.c**:

Implementiamo SYS_WRITE

```
int sys_write(int fd, userptr_t buf, size_t len) {
    if (fd != 1 && fd != 2) {
        kprintf("Sys write vuole output e stderr");
        return -1;
    }
    char *p = (char*) buf;
    for(size_t i = 0; i < len; i++) {
        putchar(p[i]);
    }
    return (int) len;
}
```

Prende come parametri:

- fd → file descriptor (per identificare dove scrivere: STDOUT, STDERR)
- buf_ptr → puntatore a spazio user (dove viene chiamata la system call), contiene i dati da scrivere
- size → dimensione dati da scrivere

Per prima cosa, se fd != 1 (STDOUT), o fd != 2 (STDERR) si ritorna errore. Poi per ogni byte da scrivere si chiama **putch**(byte[i]) che equivale a fare kprintf("%c", p[i]), NON kprintf("&s", p).

SYS_READ

Se un programma user chiama read(fd, buf, nb) la destinazione della lettura (buf) deve essere un indirizzo dello spazio user, non del kernel, quindi nel caso l'errore deve essere gestito correttamente.

```
int sys_read(int fd, userptr_t buf, size_t len) {
    if (fd != 0) {
        kprintf("Sys read accetta sta in input");
        return -1;
    }
    char *p = (char*) buf;
    for(size_t i = 0; i < len; i++) {
        p[i] = getch();
    }
    return (int) len;
}
```

Stessi parametri della SYS_WRITE(fd → file descriptor del file aperto, buf → buffer dello user space dove verranno copiati i dati letti, size = numero max di byte da leggere). Se fd != 0 (STDIN) abbiamo errore, ritorna -1. La funzione RITORNA il NUMERO di CARATTERI LETTI dal registro tf → tf_a0. Buf (regione di memoria utente, USER SPACE) viene aggiornato con i singoli caratteri letti da STDIN (riga di comando), tramite **getch()**. Dobbiamo anche controllare che buf non punti ad indirizzi non validi, appartenenti a kseg0 (kernel space).

La destinazione finale della read() deve essere un indirizzo appartenente a USER SPACE!!! Il programma che invoca read() si aspetta che al ritorno della funzione i dati siano disponibili nel proprio buffer della memoria utente, altrimenti non potrebbe accedere direttamente alla memoria di KERNEL SPACE.

Se **2 processi utente** (eseguiti contemporaneamente) **aprono lo stesso file in lettura ed eseguono la chiamata read(fd, V, N)** su quel file:

- **File descriptor è diverso** Il fd è semplicemente un intero usato da un processo usato per identificare un file aperto, è locale al processo. Non identifica il file

a livello globale. Identifica solo una voce nella tabella dei File Descriptor Table (file aperti di quel processo).

- I 2 processi leggono il file INDIPENDENTEMENTE l'uno dall'altro, ciascuno ha aperto il file con la sua `open()`. Ogni volta che un processo esegue `open()`, il kernel crea una nuova Open File Structure.
- NON è necessario usare un lock per fare avvenire le operazioni in mutua esclusione, perché si tratta di operazioni di lettura, non di scrittura, quindi non è necessaria alcuna sincronizzazione.
- Se l'implementazione della `read()` utilizza un kernel buffer, i 2 processi utilizzano lo stesso buffer? Dipende dall'implementazione. Se il kernel buffer viene allocato dinamicamente all'interno della system call, allora i buffer non sono condivisi, appartiene solo a quella chiamata. Altrimenti: `char kbuf[4096]`; → il kernel dichiara un buffer una sola volta e quindi entrambi accedono al kernel buffer globale / condiviso... serve sincronizzazione.

C) Given the following implementation of `sys_read`, for read operations on file

```
static int
file_read(int fd, userptr_t buf_ptr, size_t size) {
    ...
    kbuf = kmalloc(size);
    uio_kinit(&iiov, &ku, kbuf, size, of->offset, UIO_READ);
    result = VOP_READ(vn, &ku);
    if (result) {
        return result;
    }
    of->offset = ku.uio_offset;
    nread = size - ku.uio_resid;
    copyout(kbuf, buf_ptr, nread);
    kfree(kbuf);
    return (nread);
}
```

Answer the following questions:

- 1) Does the operation exploit a kernel buffer? (motivate)
 - 2) Is it possible that the return value differs from the value of size parameter? (motivate)
 - 3) Could `kmalloc` be called with a parameter $< \text{size}$ (e.g. `kmalloc (size/2)`)? (motivate)
- 1) **Sì.**
Il buffer `kbuf` è un buffer temporaneo nel **Kernel Space**, allocato dinamicamente con `kmalloc()`. Ogni chiamata a `read()` possiede il proprio buffer, quindi **non è condiviso** tra processi.
- 2) **Sì.** La funzione restituisce: `nread = size - uio_resid`; cioè il **numero effettivo di byte letti**. Può essere minore di `size`, ad esempio se: si raggiunge la fine del file (EOF); sono disponibili meno byte del richiesto.
- 3) **VOP_READ** in questo caso tenta di leggere fino a `size`. Se allocassimo un buffer temporaneo con `size/2`, si rischierebbe di scrivere oltre lo spazio allocato. Sarebbero necessarie più chiamate a `VOP_READ` per leggere tutti i `size` byte, con una sola `VOP_READ` (nell'implementazione corrente), il buffer deve avere dimensione almeno `size`. Quindi mantenendo una sola lettura la risposta è no.

`SYS_EXIT` (in `proc_syscalls.c`)

```

void
sys__exit(int status)
{
    /* get address space of current process and destroy */
    struct addrspace *as = proc_getas();
    as_destroy(as);
    /* thread exits. proc data structure will be lost */
    thread_exit();

    panic("thread_exit returned (should not happen)\n");
    (void) status; // TODO: status handling
}

```

`sys__exit()` è la **system call che termina il processo corrente**, invocata quando un programma esegue `exit(status)`, dove **status** rappresenta il codice di terminazione del processo. Per prima cosa recupera l'**Address Space** del processo corrente tramite `proc_getas()`: questa struttura contiene tutta la memoria virtuale del processo, cioè segmento codice, segmento dati, heap e user stack. Successivamente `as_destroy(as)` distrugge l'Address Space, liberando tutta la memoria e le strutture associate al processo. Terminata la deallocazione della memoria, viene chiamata `thread_exit()`, che termina il thread corrente; poiché in questa versione di OS/161 ogni processo possiede un solo thread, la terminazione del thread comporta anche la terminazione del processo. `thread_exit()` è una funzione che **non deve mai ritornare**: se ciò accadesse, significherebbe che si è verificato un grave errore nel kernel e per questo viene eseguita `panic()`, che arresta il sistema. Infine, l'istruzione `(void) status` serve unicamente ad eliminare il warning del compilatore, poiché in questa implementazione semplificata il parametro `status` non viene ancora utilizzato.

PCB (Process Control Block) di OS161: struct proc

```

struct proc {
    char *p_name;
    struct spinlock p_lock;
    unsigned p_numthreads;

    struct addrspace *p_addrspace;

    struct vnode *p_cwd;

    ...
};

```

Struttura del kernel che rappresenta un processo e contiene le **risorse condivise** da tutti i thread appartenenti al processo, tra cui: nome del processo, lock, numero di thread (`p_numthreads`), **Address Space** (`p_addrspace`) e **Current Working Directory** (`p_cwd`). Per eseguire un programma utente (ad esempio `p_testbin/palin`), il menu di OS/161 chiama `proc_create_runprogram()`, che crea la struct `proc`, e `thread_fork()`, che crea il thread kernel associato. Il nuovo thread esegue `cmd_progtread()`, che invoca `runprogram()`: questa funzione crea

l'Address Space, carica il file eseguibile **ELF** e, tramite `enter_new_process()`, trasferisce il thread dalla **Kernel Mode** alla **User Mode**, iniziando l'esecuzione del programma utente.

RUNPROGRAM

```

int
runprogram(char *programe)
{
    struct addrspace *as;
    struct vnode *v;
    vaddr_t entrypoint, stackptr;
    int result;

    result = vfs_open(programe, O_RDONLY, 0, &v);

    as = as_create();

    proc_setas(as);
    as_activate();
}

```

```

result = load_elf(v, &entrypoint);

vfs_close(v);

result = as_define_stack(as, &stackptr);

enter_new_process(0,
                  NULL,
                  NULL,
                  stackptr,
                  entrypoint);

panic("enter_new_process returned\n");
return EINVAL;

```

runprogram() è la funzione che **avvia un nuovo programma utente**. Riceve come parametro il **nome del file eseguibile**. Le principali variabili utilizzate sono: as, che rappresenta il nuovo **Address Space** del processo; vnode, che identifica il file eseguibile aperto nel VFS; entrypoint, che conterrà l'indirizzo virtuale della prima istruzione del programma; e stackptr, che conterrà il valore iniziale dello **User Stack Pointer**. Per prima cosa il kernel apre il file eseguibile in sola lettura tramite vfs_open(). Successivamente as_create() crea un nuovo **Address Space** inizialmente vuoto, che viene associato al processo corrente con proc_setas(as) e attivato sulla CPU con as_activate(). La funzione load_elf() legge il file ELF, carica nell'Address Space i segmenti del programma (codice, dati, ecc.) e restituisce l'**entrypoint**, cioè l'indirizzo della prima istruzione da eseguire. Terminato il caricamento, il file viene chiuso con vfs_close(). Successivamente as_define_stack() inizializza lo **User Stack** e restituisce il valore iniziale dello **Stack Pointer** (stackptr). Infine enter_new_process() prepara il contesto iniziale del processo e realizza il passaggio dalla **Kernel Mode** alla **User Mode**, impostando **PC = entrypoint**, **SP = stackptr** e avviando l'esecuzione del programma. Questa funzione **non ritorna**, perché da quel momento la CPU continua l'esecuzione direttamente nel programma utente.

ENTER_NEW_PROCESS

```

void enter_new_process(int argc, userptr_t argv, userptr_t env, vaddr_t stack, vaddr_t entry)
{
    struct trapframe tf;

    bzero(&tf, sizeof(tf));

    tf.tf_status = CST_IRQMASK | CST_IUp | CST_KUp;

    tf.tf_epc = entry;

    tf.tf_a0 = argc;

    tf.tf_a1 = (vaddr_t)argv;

    tf.tf_a2 = (vaddr_t)env;

    tf.tf_sp = stack;

    mips_usermode(&tf);
}

```

enter_new_process() prepara il **contesto iniziale della CPU** e trasferisce definitivamente il thread dalla **Kernel Mode** alla **User Mode**. Per farlo costruisce un **Trapframe**, una struttura che rappresenta lo

stato dei registri MIPS. Il trapframe viene inizializzato a zero e configurato impostando il **Program Counter (PC)** all'entrypoint (prima istruzione del programma ELF), lo **Stack Pointer (SP)** a stackptr e i registri degli argomenti (a0, a1, a2) con argc, argv ed env. Infine mips_usermode() passa il trapframe al codice assembly, che ripristina i registri della CPU ed entra in **User Mode**, avviando l'esecuzione del programma.

Trapframe normale vs enter_new_process()

- **Trap normale:** un programma è già in esecuzione, entra nel kernel tramite una system call o una trap e la CPU salva automaticamente il suo stato nel **Trapframe**. Terminata la system call, il trapframe viene ripristinato e il programma riprende dal punto in cui era stato interrotto.
- **enter_new_process():** il programma non è ancora mai stato eseguito, quindi non esiste alcun trapframe. Il kernel ne costruisce uno manualmente con lo stato iniziale del processo e utilizza lo stesso meccanismo di **return from exception** per entrare in modalità utente.

Il **Trapframe** salva il contesto del **programma utente** (registri, PC e SP) durante il passaggio al kernel, mentre lo **Switchframe** salva il contesto del **thread kernel** (registri kernel, SP e RA) durante il **context switch**. Ogni processo possiede un **Address Space** indipendente (codice, dati, heap e user stack), mentre ogni thread possiede un **Kernel Stack** indipendente.

STRUTTURA addrspace:

```
struct addrspace {  
  
    vaddr_t as_vbase1;  
    paddr_t as_pbase1;  
    size_t  as_npages1;  
  
    vaddr_t as_vbase2;  
    paddr_t as_pbase2;  
    size_t  as_npages2;  
  
    paddr_t as_stackbase;  
};
```

La **struct addrspace** descrive lo **spazio di indirizzamento** del processo. Non contiene direttamente i dati del programma, ma descrive dove sono memorizzati.

Nella versione dumbvm comprende:

- **Prima regione** (generalmente il segmento codice): as_vbase1 (base virtuale), as_pbase1 (base fisica in RAM) e as_npages1 (numero di pagine).
- **Seconda regione** (generalmente dati): as_vbase2, as_pbase2 e as_npages2.
- **Stack:** as_stackbase, che rappresenta la base fisica dello **User Stack** (lo stack virtuale si trova nella parte alta dello spazio di indirizzamento).

Programmazione concorrente

Un **thread** è il **flusso di esecuzione** di un programma. Ogni thread possiede un proprio **contesto di esecuzione**, costituito dallo stato della CPU (**Program Counter, Stack Pointer, registri e modalità di esecuzione**) e dal proprio **stack**.

La memoria di un processo comprende:

- **Program Code** (codice);
- **Program Data** (variabili globali);
- **Heap**;
- **Stack**.

Codice, dati e heap appartengono al **processo**, mentre il **contesto di esecuzione** appartiene al **thread**.

Esistono due tipi di thread:

- **User Thread**, gestiti dalla libreria utente;
- **Kernel Thread**, gestiti dal kernel.

Gli **User Thread** non possono essere eseguiti direttamente dalla CPU, ma devono essere **mappati su Kernel Thread**. I principali modelli di mapping sono:

- **One-to-One**: un User Thread ↔ un Kernel Thread.
- **Many-to-One**: più User Thread ↔ un solo Kernel Thread (nessun parallelismo).
- **Many-to-Many**: più User Thread ↔ più Kernel Thread (parallelismo).

Un processo può contenere **più thread**, che **condividono codice, dati, heap e file aperti**, ma ciascuno mantiene **Program Counter, registri e stack propri**.

Si ha una struct thread:

```
/* see kern/include/thread.h */

struct thread {
    char *t_name;                /* Name of this thread */
    const char *t_wchan_name;    /* Name of wait channel, if sleeping */
    threadstate_t t_state;       /* State this thread is in */

    /* Thread subsystem internal fields. */
    struct thread_machdep t_machdep;
    struct threadlistnode t_listnode;
    void *t_stack;               /* Kernel-level stack */
    struct switchframe *t_context; /* Saved register context (on stack) */
    struct cpu *t_cpu;           /* CPU thread runs on */
    struct proc *t_proc;         /* Process thread belongs to */
    ...
};
```

Il TCB (Thread Control Block) è la struttura dati usata per memorizzare il contesto di un thread. Questa è rappresentata dalla **struct thread**. **Contiene: name, stato del thread corrente (wait, sleeping, running), primitive di sincronizzazione (wchan_name), puntatore a CPU su cui è in esecuzione il thread corrente, puntatore allo kernel stack (t_stack), puntatore a contesto del thread (t_context), rappresentato da SWITCHFRAME, puntatore a processo a cui appartiene il thread (t_proc).**

Durante un **context switch**, il kernel salva il contesto del thread corrente nel suo **Switchframe** e ripristina quello del thread successivo.

In kern/thread/thread.c abbiamo

THREAD_FORK

Crea un nuovo kernel thread.

```
int thread_fork(const char *name, struct proc *proc, void (*entrypoint)(void *, unsigned long), void *data1, unsigned long data2);
```

PARAMETRI:

- Name = nome thread
- proc = se proc != NULL so già a quale processo appartiene, se proc = NULL usa automaticamente il processo corrente, fallo appartenere al processo del thread che sta chiamando thread_fork().
- Entrypoint → funzione che eseguirà il thread.
- Data1, data2 → parametri passati alla funzione.

La funzione restituisce **0** in caso di successo oppure un **codice di errore (errno)**. Non restituisce un puntatore a struct thread perché thread_fork è pensata come API di alto livello, la struct è un dettaglio interno del kernel. Al termine di thread_fork() il thread è pronto per essere eseguito ma non sta ancora usando la CPU.

Dentro contiene:

newthread = thread_create(...); → *alloca la struttura struct thread tramite kmalloc(sizeof(struct thread)); Vengono inizializzati i campi principali: nome, stack, stato, PID del processo, contest e alla fine otteniamo un oggetto thread*

switchframe_init(newthread, entrypoint, data1, data2); → *inizializza il contesto iniziale del thread, prepara lo stack come se il thread fosse già stato interrotto una volta. Vengono salvati PC iniziale (entrypoint), parametri dell'entrypoint data1 e data2, registri iniziali. Questa struttura prende il nome di switchframe ed è quella che userà il contextswitch.*

thread_make_runnable(newthread, false) → *cambia lo stato da S_NEW a S_READY (target->t_state = S_READY), inserisce il thread nella Run Queue (threadlist_addtail(&targetcpu->c_runqueue, target);. Il thread è pronto ma non sta ancora eseguendo. Sarà lo scheduler a decidere quando eseguirlo: thread_switch()*

```
int thread_fork(const char *name, struct proc *proc, void (*entrypoint)(void *, unsigned long), void *data1, unsigned long data2) {
```

```
    struct thread *newthread;
```

```
    newthread = thread_create(name); /* Crea la struttura del thread */
```

```
    if (proc == NULL) /* Se proc è NULL usa il processo corrente */
```

```
        proc = curproc;
```

```
    proc_addthread(proc, newthread); /* Associa il thread al processo */
```

```
    switchframe_init(newthread, entrypoint, data1, data2); /* Prepara il contesto iniziale del thread */
```

```
    thread_make_runnable(newthread, false); /* Rende il thread eseguibile */
```

```
    return 0; }
```

Alla fine di `thread_fork()` il nuovo thread è stato creato, inizializzato, messo in stato `S_READY`, inserito nella Run Queue, ma non sta ancora usando la CPU quindi:

`THREAD_SWITCH` → effettua il context switch (passaggio da thread Corrente a un altro thread pronto nella run queue).

```
void thread_switch(threadstate_t newstate, ...)
{
    struct thread *cur, *next;  (2 puntatori: il primo indica il thread che sta usando attualmente la CPU, il secondo il prossimo thread da eseguire)

    cur = curthread; /* Thread attualmente in esecuzione */

    switch (newstate) {        /* Aggiorna lo stato del thread corrente */
        case S_RUN:
            panic("Illegal S_RUN in thread_switch\n");

        case S_READY:
            thread_make_runnable(cur, true);        //true = possiedo già il lock per lavorare sulla run queue. (in thread_fork
            avevamo invece false).

            break;                /* Altri casi: S_SLEEP, S_ZOMBIE... */

    }

    next = threadlist_remhead(&curcpu->c_runqueue);    /* Seleziona il prossimo thread pronto */

    switchframe_switch(&cur->t_context, &next->t_context);    /* Esegue il context switch, viene fatto in assembly
    perché riguarda direttamente i registri della CPU */

}
```

`thread_switch()` realizza il **context switch**, cioè il passaggio della CPU dal thread corrente ad un altro thread pronto. Per prima cosa recupera il thread corrente (`curthread`) e aggiorna il suo stato in base al parametro `newstate`.

- Se `newstate == S_READY`, il thread cede volontariamente la CPU (ad esempio dopo `thread_yield()`): `thread_make_runnable()` lo reinserisce nella **Run Queue**.
- Se `newstate == S_RUN`, viene generato un panic, perché un thread che sta lasciando la CPU non può rimanere nello stato **RUNNING**.

Successivamente `threadlist_remhead()` estrae il primo thread pronto dalla **Run Queue** e `switchframe_switch()` esegue il vero **context switch**, salvando i registri della CPU nel `t_context` del thread corrente e ripristinando quelli del thread successivo.

Dopo il cambio di contesto possono verificarsi due situazioni:

- **Thread già eseguito** → riprende dall'istruzione in cui era stato sospeso.
- **Thread appena creato** → inizia l'esecuzione dall'entrypoint preparato da `switchframe_init()`.

Ci sono dei TEST che mette a disposizione OS161 per verificare che il sottosistema di thread funzioni correttamente.

Test	Comando (dal menu OS/161)	Cosa esegue	A cosa serve

tt1	OS/161> tt1	Richiama threadtest -> runthreads(1) . Vengono creati 8 thread (NTHREADS = 8) tramite thread_fork() , che eseguono la funzione loudthread() . Ogni thread stampa circa 120 caratteri , producendo un output mescolato.	Verifica che thread_fork() , lo scheduler e il context switch (thread_switch()) funzionino correttamente. Se l'output è interleavato, significa che i thread si stanno alternando sulla CPU.
tt2	OS/161> tt2	Richiama threadtest2 -> runthreads(0) . Vengono creati 8 thread che eseguono quietthread() . Ogni thread esegue un busy wait (circa 200000 iterazioni) e poi stampa un solo carattere (0...7).	Verifica che lo scheduler gestisca correttamente thread che eseguono molto lavoro e che tutti i thread terminino correttamente , senza perdere o bloccare nessun thread.
tt3	OS/161> tt3	Richiama threadtest3 -> runtest3() . Crea un insieme di thread che eseguono operazioni di sleep , lavoro e sincronizzazione utilizzando primitive del kernel (lock, semafori, condition variable, ecc.).	Verifica il corretto funzionamento delle primitive di sincronizzazione e della gestione degli stati dei thread (RUN, READY, SLEEP, ecc.), controllando che i thread possano bloccarsi e risvegliarsi correttamente.

In kern/test/threadtest.c abbiamo la funzione di test **runthreads(int doloud)** che crea un certo numero di thread.

```
void runthreads(int doloud)
```

```
{
    for (int i = 0; i < NTHREADS; i++) {
        result = thread_fork("threadtest", NULL, doloud ? loudthread : quietthread, NULL, i);
    }
    for (int i = 0; i < NTHREADS; i++) {
        P(tsem);
    }
}
```

Il test crea **8 kernel thread** tramite **thread_fork()**. In base al valore di **doloud**, ogni thread esegue la funzione **loudthread()** oppure **quietthread()**. Il parametro **NULL** indica che il thread appartiene al **processo corrente**, mentre **i** identifica il thread. Il thread principale deve attendere la terminazione di tutti gli 8 thread. Per questo utilizza un **semaforo**:

- **P(tsem)** → il thread principale si blocca in attesa.
- **V(tsem)** → ogni thread, al termine della propria esecuzione, segnala la propria conclusione.

Dopo aver ricevuto **8 segnali**, il test termina. Al termine di **thread_fork()** il thread: viene creato; viene inizializzato con il proprio **Switchframe**; viene posto nello stato **S_READY**; viene inserito nella **Run Queue**. Il thread **non esegue immediatamente**: sarà lo scheduler a selezionarlo.

thread_switch() realizza il **context switch**, selezionando un thread pronto dalla **Run Queue**.

La funzione **switchframe_switch()** esegue il vero cambio di contesto:

- salva il contesto del thread corrente (**cur->t_context**);
- ripristina il contesto del thread successivo (**next->t_context**);
- il nuovo thread riprende l'esecuzione dal punto in cui era stato sospeso oppure, se è appena stato creato, inizia dall'entrypoint preparato da **switchframe_init()**.

Il **context switch** consiste nel **salvare e ripristinare i registri della CPU**, operazione implementata in **assembly** perché coinvolge direttamente l'hardware. Il contesto salvato nello **Switchframe** comprende principalmente **Program Counter (PC)**, **Stack Pointer (SP)** e i registri salvati del processore.

- **thread_yield()** → permette al thread corrente di **cedere volontariamente la CPU**, senza terminare e senza bloccarsi. Il thread passa: `S_RUN` → `S_READY` viene reinserito nella **Run Queue** e potrà essere eseguito di nuovo in seguito. Internamente richiama il cambio di contesto con il nuovo stato `S_READY`.
- **thread_exit()** → termina definitivamente il thread corrente. Rimuove il thread dal processo; portarlo in uno stato terminale, tipicamente `S_ZOMBIE`; eseguire un context switch verso un altro thread. Il thread non viene distrutto immediatamente, perché sta ancora usando il proprio **Kernel Stack**. La funzione non deve mai ritornare.
- **thread_destroy()** → libera materialmente le risorse di un thread terminato (kernel stack, nome, struct thread). Non può essere eseguita dal thread stesso perché distruggerebbe lo stack sul quale sta ancora eseguendo, quindi viene chiamata successivamente da un altro thread, dopo il context switch. Fa tutte `kfree(t_name, t_stack...)`. Ci sono vari `KASSERT`:
 - `KASSERT(thread != curthread)`
 - `KASSERT(thread -> t_state != S_RUN)` (deve essere in stato `S_ZOMBIE`)
 - ...

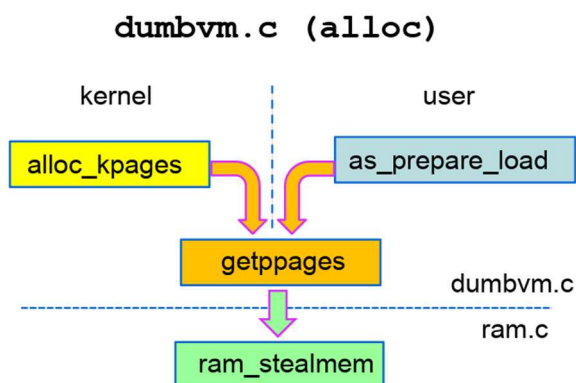
Il programma viene caricato nell'address space da un file eseguibile in formato ELF.

La system call `execv()` : `int execv(const char *program, char **args)` → viene invocata da un processo utente già esistente, non crea un nuovo processo. Il suo scopo è sostituire completamente il programma attualmente in esecuzione con un altro. Program rappresenta il percorso del file ELF, invece args contiene gli argomenti della riga di comando che verranno passati al nuovo programma. Il PID, quando viene eseguita `execv()`, rimane invariato. Il kernel elimina il vecchio address space, ne crea uno nuovo, carica il file, ricrea lo stack e infine inizia l'esecuzione del nuovo programma dalla sua prima istruzione. `Fork()` invece crea un nuovo processo, duplicando quello esistente. Il file ELF contiene text segment e data segment, ma non stack segment.

Memoria

OS161 suddivide la memoria in PAGINE di dimensione fissa (4KB). I frame vengono allocati sempre in posizioni CONTIGUE della RAM (quindi abbiamo allocazione contigua della memoria fisica + paginazione). Dumbvm è la più semplice implementazione della memoria virtuale di OS161.

ALLOCAZIONE MEMORIA



KERNEL: richiede memoria tramite **kmalloc()** che utilizza **alloc_kpages()**. Questa invoca **getppages()** che a sua volta chiama **ram_stealmem()**. Kmalloc NON utilizza una partizione fisica diversa da quella dei processi utente, kernel e processi attingono alla stessa memoria fisica disponibile con meccanismi che si basano su **ram_stealmem()**

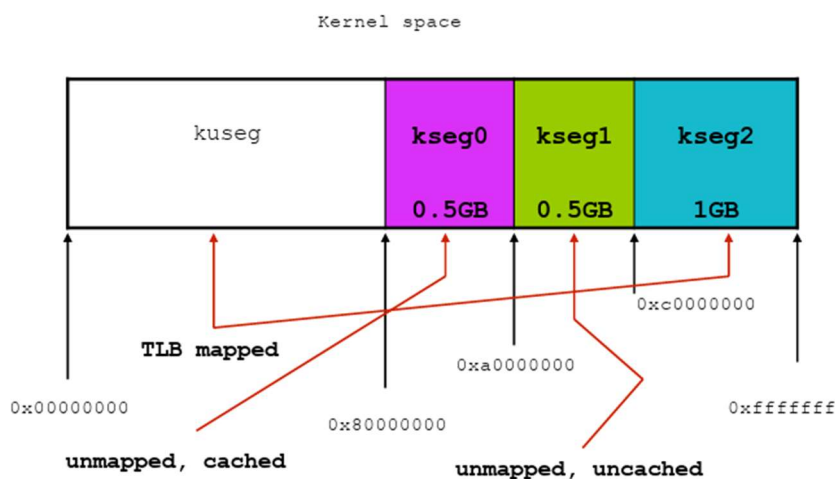
Dumbvm.c → alto livello

Ram.c → basso livello (architettura del sistema)

Getppages() richiede pagine di memoria, invece ram_stealmem() le alloca partendo dal **firstpaddr = primo indirizzo fisico libero**.

Lo spazio di indirizzamento è comune al kernel e allo user: il kernel può vedere nello spazio di indirizzamento del processo user ma non viceversa.

MIPS VIRTUAL ADDRESS SPACE



In OS/161, user programs live in kuseg, kernel code and data structures live in kseg0, devices are accessed through kseg1, and kseg2 is not used.

INDIRIZZO FISICO = INDIRIZZO LOGICO – 0x80000000

TOTALE spazio di indirizzamento logico = 4GB (indirizzi da 32 bit):

- 2GB user space = **kuseg**
- **2GB kernel space** diviso in : **kseg0** = spazio kernel (data, codice, stack) (TLB unmapped, cached); **kseg1** = dispositivi I/O memory mapped, uncached (non si può utilizzare la cache per leggere / scrivere su dispositivi di I/O); **kseg2** = non utilizzato.

I dispositivi IO devono essere mappati nello spazio del kernel. Sono mappati su kseg1 perché non è mappato in cache e un dispositivo IO non può essere letto/scritto utilizzando la cache perché qualunque operazione deve essere fatta direttamente sul dispositivo stesso. Non può essere mappato su kseg0 perché è mappato in cache.

INDIRIZZI LOGICI USER = 0x00000000 – 0x7fffffff (parte RAM nominata KUSEG)

INDIRIZZI LOGICI KERNEL = 0x80000000 (MIPS_KSEG0) – 0xffffffff

Primo indirizzo della memoria fisica = 0x00000000

Questi due indirizzi (FIRSTFREE e FIRSTPADDR) rappresentano **la stessa posizione della memoria**, ma in due spazi di indirizzamento diversi.

Nome	Tipo	Esempio	Significato
firstfree	Indirizzo virtuale	0x8003B000	Primo indirizzo libero nello spazio kernel (KSEG0), dopo aver allocato il kernel
firstpaddr	Indirizzo fisico	0x3B000	Primo frame fisico libero della RAM.

Lo stack del thread kernel viene allocato subito dopo il kernel.

Allochiamo da firstpaddr a ramsize.

MAPPATURA PROCESSI UTENTE

- **FISICO = (virtuale – vbase) + pbase**
- **VIRTUALE = (fisico – pbase) + vbase**

Il sistema di allocazione è comune tra kernel e user space !!!

Si parte da **ram_bootstrap()** che viene eseguita durante l'avvio del kernel e determina l'intervallo di RAM disponibile [firstpaddr, lastpaddr) che può essere allocata.

```
void
ram_bootstrap(void) {
    /* Get size of RAM. */
    size_t ramsize = mainbus_ramsize();
    if (ramsize > 512*1024*1024) {
        ramsize = 512*1024*1024;
    }
    lastpaddr = ramsize;
    /* Get first free virtual address from where
       start.S saved it. Convert to physical address. */
    firstpaddr = firstfree - MIPS_KSEG0;
}
```

Inizializza:

- firstpaddr → primo indirizzo fisico libero
- lastpaddr → indirizzo finale della RAM

Dimensione RAM = mainbus_ramsize()

Firstfree = indirizzo virtuale in kseg0 = 0x8003B000

Poi riporta la conversione da indirizzo logico a fisico (dato che kseg0 è direttamente mappato sulla RAM). Questa funzione si limita a inizializzare i confini che verranno usati da ram_stealmem(npages).

L'allocatore vero e proprio si trova in kern/arc/mips/vm/ram.c e si chiama **ram_stealmem()** (ruba memoria alla RAM). Alloca incrementalmente dopo il kernel (che è già caricato nella parte iniziale della RAM). Non distingue formalmente le pag kernel da quelle dei processi (non impedisce interleaving delle allocazioni kernel e utente). DUMBVM utilizza la TLB. Quando avviene un TLB fault, vm_fault determina la corrispondenza tra pagina virtuale e fisica e inserisce la traduzione nella

TLB. Lo stack utente viene collocato in alto nello spazio virtuale, ma NON viene allocato alla fine della memoria fisica. Lo stack kernel non è globale, uno per ogni kernel thread.

Una bitmap non usa partizioni di dimensione diversa per kernel e processi

```
paddr_t ram_stealmem(unsigned long npages) {
    paddr_t paddr;
    size_t size = npages * PAGE_SIZE;

    if (firstpaddr + size > lastpaddr) {
        return 0;
    }

    paddr = firstpaddr;
    firstpaddr += size;

    return paddr;
}
```

PARAMETRO IN INGRESSO = npages → numero di pag fisiche contigue richieste. Quindi la dimensione totale richiesta è npages * PAGE_SIZE.

VALORE DI RITORNO = paddr_t → indirizzo fisico iniziale del blocco assegnato. Restituisce 0 se la RAM non è sufficiente (if).

Firstpaddr è il puntatore al primo indirizzo libero in memoria e lo faccio avanzare alla prima zona fisica mai utilizzata.

La memoria viene rubata dalla RAM libera, non viene recuperata perché non si tiene traccia delle partizioni allocate e quindi non si può deallocare.

In kern/arc/mips/vm/dumbvm.c abbiamo **getppages**

```
static paddr_t
getppages(unsigned long npages) {
    paddr_t addr;

    spinlock_acquire(&stealmem_lock);

    addr = ram_stealmem(npages);

    spinlock_release(&stealmem_lock);

    return addr;
}
```

E' una funzione interna di dumbvm.c che richiede un blocco di npages pagine fisiche contigue e restituisce l'indirizzo fisico iniziale del blocco.

Static → indica che la funzione è visibile solo all'interno del file dumbvm.c, non può essere chiamata direttamente da altri file del kernel.

La spinlock protegge la chiamata a ram_stealmem() ed è necessaria perché ram_stealmem() legge e modifica la variabile globale firstpaddr. Senza lock 2 thread potrebbero leggere contemporaneamente lo stesso valore di firstpaddr e ricevere lo stesso blocco di memoria. Poi il lock viene rilasciato. Sia la memoria dinamica del kernel sia la memoria dei processi utente finiscono per usare questa interfaccia

ed allocare tramite `ram_stealmem()`. `Getppages()` sa allocare memoria, ma nella versione originale non esiste un meccanismo per rilasciarla (`ram_stealmem()` fa avanzare il puntatore `firstpaddr`, ma la memoria non viene mai restituita, quando il processo termina le pag rimangono inutilizzabili e diminuisce la RAM disponibile).

Come viene svolta la deallocazione della memoria:

- soluzione 1) gestione completa in `ram.c`: si realizzano delle funzioni analoghe a `getppages()` (`freeppages()`) e `ram_stealmem()` (`ram_freemem()`) inerenti alla deallocazione.
Dovremmo modificare il sottosistema RAM originale
- soluzione 2) gestione in `dumbvm.c`: quella usata da noi →

`getppages()` ↔ `freeppages()`

↓

`ram_stealmem()`

`ram_stealmem()` continua a gestire la RAM non utilizzata e `dumbvm.c` mantiene una tabella dei frame liberati.

- Si utilizza allocazione contigua per kernel e pagine per user, ma non fattibile (si implementa un sistema di paging da 0).

Nel nostro caso: modifichiamo `dumbvm.c` ma non cambiamo `ram.c`. Introduciamo una struttura dati: **`freeRamFrames`** e **`allocSize`** che tengono traccia delle pag fisiche liberate. Quando un processo termina, `as_destroy()` richiama `freeppages()` che segna i frame come liberi nella tabella. Quando arriverà una nuova richiesta di memoria, `getppages()` non chiamerà subito `ram_stealmem()` ma prima proverà a trovare un blocco di pag già liberato con `getfreeppages()`. Se trova un blocco sufficientemente grande, allora lo riutilizza. Solo che non esistono pag libere disponibili si ricorre a `ram_stealmem()` che alloca nuova memoria fisica. Quindi la memoria dei processi terminati viene riciclata.

VARIABILI GLOBALI NECESSARIE:

- Static struct `spinlock freemem_lock = SPINLOCK_INITIALIZER` → per proteggere le nuove strutture dati, visto che + thread potrebbero allocare /liberare memoria contemporaneamente (chiamare `alloc_kpages()` e `free_kpages()`)
- static unsigned char **`*freeRamFrames = NULL;`** → **BITMAP**: ogni element rappresenta un frame fisico della RAM. 0 = occupato, 1 = libero. Contiene un numero di elementi pari al numero di frame. E' un puntatore ad un array di unsigned char, per sapere rapidamente quali frame possono essere allocati.
- static unsigned long **`*allocSize`** = NULL → vettore parallelo a `freeRamFrames`. Indica quanto è grande ogni partizione allocate. Ad es se abbiamo `allocSize[10] = 4` → il blocco che parte dal frame 10 occupa 4 pag consecutive. Puntatore ad array `allocSize[i]` = numero di pag allocate a partire dall'indirizzo i-esimo.
- static int **`nRamFrames`** = 0 → numero totale di frame presenti nella RAM (RAM size / 4KB)
- static int **`allocTableActive`** = 0 → variabile booleana che indica se la tabella di allocazione è già stata inizializzata. Se vengono creati `freeRamFrames` e `allocSize`, viene settata a 1.

Frame indice	0	1	2	3	4	5	6	7	8	9	10	11
RAM	A	A	A	A	A	F	F	A	A	A	A	F
freeRamFrames	0	0	0	0	0	1	1	0	0	0	0	1
allocSize	3	0	0	2	0	0	0	4	0	0	0	0

Prima dell'inizializzazione delle tabelle, la memoria veniva ottenuta con `ram_stealmem()` che non supporta la deallocazione, quindi tutti i frame assegnati in questa fase restano occupati per tutta la vita del kernel. Dopo la creazione delle tabelle, entra in funzione il nuovo allocatore basato su `freeRamFrames` e `allocSize`. Da qui ogni chiamata ad `alloc_kpages()` registra l'allocazione nelle tabelle e `free_kpages()` è in grado di liberare i frame che diventano disponibili.

INIZIALIZZAZIONE: VM_BOOTSTRAP

Viene eseguita durante l'avvio del kernel e costruisce le strutture necessarie per il nuovo sistema di gestione della memoria.

```
void vm_bootstrap(void) {
    int i;
    nRamFrames = ((int)ram_getsize())/PAGE_SIZE;
    /* alloc freeRamFrame and allocSize */
    freeRamFrames = kmalloc(sizeof(unsigned char)*nRamFrames);
    allocSize      = kmalloc(sizeof(unsigned long)*nRamFrames);
    if (freeRamFrames==NULL || allocSize==NULL) {
        /* reset to disable this vm management */
        freeRamFrames = allocSize = NULL; return;
    }
    for (i=0; i<nRamFrames; i++) {
        freeRamFrames[i] = (unsigned char)0; allocSize[i] = 0;
    }
    spinlock_acquire(&freemem_lock);
    allocTableActive = 1;
    spinlock_release(&freemem_lock);
}
```

`Ram_getsize()` → mi da la dimensione della RAM fisica. Dividendo per `PAGE_SIZE` ottengo il numero totale di frame fisici (`nRamFrames`). Poi allochiamo le 2 tabelle `freeRamFrames` e `allocSize`. Se una delle 2 `kmalloc()` fallisce il nuovo sistema viene disabilitato e la funzione termina. Poi inizializziamo tutto a 0 = OCCUPATO. All'inizio infatti nessun frame viene considerato riutilizzabile. Infine si segnala che le tabelle sono pronte: `allocTableActive = 1;`

GETPPAGES

```
static paddr_t getppages(unsigned long npages) {
    paddr_t addr;

    /* try freed pages first */
    addr = getfreeppages(npages);
    if (addr == 0) { /* call stealmem */
        spinlock_acquire(&stealmem_lock);
        addr = ram_stealmem(npages);
        spinlock_release(&stealmem_lock);
    }
    if (addr != 0 && isTableActive()) {
        spinlock_acquire(&freemem_lock);
        allocSize[addr/PAGE_SIZE] = npages;
        spinlock_release(&freemem_lock);
    }
    return addr;
}
```

Parametro → npages (numero di pagine fisiche contigue richieste)

Valore di ritorno → paddr_t = indirizzo fisico iniziale del blocco trovato.

Static → la funzione è visibile solo in questo file sorgente.

La modifica principale rispetto al DumbVM originale è : `addr = getfreeppages(npages)`, quindi prova prima a recuperare le pagine che erano state allocate in precedenza e poi liberate (quelle disponibili sono quelle con `freeRamFrames[i] = 1`). Se non trova pag libere (`addr == 0`) allora `getfreeppages()` non ha trovato un blocco libero abbastanza grande, quindi si usa il vecchio sistema con `ram_stealmem(npages)`. Si usa `stealmem_lock` per impedire che 2 thread eseguano contemporaneamente `ram_stealmem()`. Altrimenti se abbiamo memoria disponibile (e le tabelle `allocSize` e `freeRamFrames` sono state già inizializzate), allora prendi il lock (`freemem_lock`) e usa `allocSize`. Questo è indicizzato con il numero del frame, quindi facciamo `addr/PAGE_SIZE` per convertire un indirizzo fisico in un numero di frame. Questa info di `allocSize` servirà dopo a `free_kpages()` per sapere quante pag liberare.

GET_FREE_PPAGES

```
static paddr_t getfreeppages(unsigned long npages) {
    paddr_t addr;
    long i, first, found, np = (long)npages;

    if (!isTableActive()) return 0;
    spinlock_acquire(&freemem_lock);
    // Linear search of free interval
    for (i=0, first=found=-1; i<nRamFrames; i++) {
        if (freeRamFrames[i]) {
            if (i==0 || !freeRamFrames[i-1])
                first = i; /* set first free in an interval */
            if (i-first+1 >= np)
                found = first;
        }
    }

    if (found>=0) {
        for (i=found; i<found+np; i++) {
            freeRamFrames[i] = (unsigned char)0;
        }
        allocSize[found] = np;
        addr = (paddr_t) found*PAGE_SIZE;
    }
    else {
        addr = 0;
    }

    spinlock_release(&freemem_lock);

    return addr;
}
```

VALORE DI RITORNO: paddr_t → indirizzo fisico della prima pag trovata

PARAMETRO: npages = numero di pag fisiche contigue richieste

Se non trova niente ritorna 0

Addr → indirizzo fisico finale da restituire

First = indice del primo frame dell'intervallo libero che stiamo analizzando

Found = indice del primo frame del blocco libero che soddisfa la richiesta.

Per prima cosa si controlla se le tabelle allocSize e freeRamFrames sono state già inizializzate da vm_bootstrap() perché altrimenti non ha senso proseguire e conviene controllare direttamente ram_stealmem(). Da questo momento la funzione accede ad allocSize e freeRamFrames con **freemem_lock** (essendo strutture condivise da + thread e devono essere protette). Ad un certo punto si scorrono tutti i frame fisici della RAM e controlliamo se freeRamFrames[i] = 1 (libero) o freeRamFrames[i] = 0 (occupato). Se la pagina i-esima è libera allora controllo se il frame corrente è il primo frame della RAM oppure se il frame precedente era occupato. Così il frame corrente rappresenta l'inizio di un intervallo libero (first). Poi controlliamo la dimensione del blocco (i-first + 1) calcola quanti frame consecutivi liberi abbiamo trovato fino a quel momento. Found = first. Si tratta di una ricerca di tipo **last-fit** (viene preso l'ultimo blocco abbastanza grande). Infatti se è stato trovato un blocco abbastanza grande, allora segno tutte le pagine all'interno dell'intervallo trovato come OCCUPATE. Salvo il numero di pag allocate e calcolo l'indirizzo fisico di partenza, altrimenti addr = 0. In realtà non è un vero e proprio first fit perché dopo found = first non fa break, quando trova un intervallo abbastanza grande il ciclo continua a scorrere tutta freeRamFrames e poi più avanti se ne ritrova un altro abbastanza grande di intervallo aggiorna di nuovo, ma alla fine resta memorizzato l'ultimo intervallo valido trovato, non il primo.

Per essere FIRST FIT vero:

```
if (i - first + 1 >= np) {  
    found = first;  
    break;  
}
```

ho trovato il primo blocco abbastanza grande e smetto subito di cercare.

Per il BEST FIT invece bisogna anche ricordare la dimensione: tra tutti i blocchi che possono contenere npages scegli quello + piccolo.

Definiamo: long min = MAX_INT; e poi

```

for (i = 0, first = found = -1; i < nRamFrames; i++) {

    if (freeRamFrames[i]) {

        if (!freeRamFrames[i - 1])
            first = i;

        if (i == nRamFrames - 1 || !freeRamFrames[i + 1]) {

            if ((i - first + 1) >= np &&
                (i - first + 1) < min) {

                found = first;
                min = i - first + 1;
            }
        }
    }
}

```

FREEPPAGES()

```

static int freeppages(paddr_t addr, unsigned long npages){
    long i, first, np=(long)npages;
    if (!isTableActive()) return 0;
    first = addr/PAGE_SIZE;
    KASSERT(allocSize!=NULL);
    KASSERT(nRamFrames>first);

    | spinlock_acquire(&freemem_lock);
    for (i=first; i<first+np; i++) {
        freeRamFrames[i] = (unsigned char)1;
    }
    spinlock_release(&freemem_lock);

    return 1;
}

```

PARAMETRI : paddr_t addr (indirizzo fisico iniziale del blocco da liberare), npages (numero di pag fisiche da liberare. Il valore di ritorno è un int (0 = operazione non riuscita, 1 = pag marcate come libere).

Per prima cosa controlliamo se le tabelle di gestione della memoria sono attive. Se non lo sono termina subito. Poi convertiamo l'indirizzo fisico in indice del frame. Poi verifichiamo che la tabella allocSize esista e che il frame iniziale sia effettivamente interno alla RAM. Acquisiamo il lock, segniamo come liberi i frame, rilasciamo il lock e ritorniamo 1. Funzione usata per liberare le pag **lato KERNEL**

AS_DESTROY()

Usata per liberare lo spazio di indirizzamento di un **processo UTENTE**.

```

void
as_destroy(struct addrspace *as)
{
    dumbvm_can_sleep();

    freepages(as->as_pbase1, as->as_npages1);
    freepages(as->as_pbase2, as->as_npages2);
    freepages(as->as_stackbase, DUMBVM_STACKPAGES);

    kfree(as);
}

```

Riceve in ingresso un puntatore allo spazio di indirizzamento del processo che deve essere distrutto e non restituisce niente. Prima controlliamo di trovarci in un contesto in cui il kernel può eseguire operazioni che possono dormire / bloccarsi. Poi si chiama 3 volte `freepages()` la prima per liberare le pagine fisiche associate al segmento di codice, poi di data, poi di stack. La dimensione dello stack è fissa. Poi `kfree(as)` libera la struttura dati stessa che descrive l'address space.

Quando termina un PROCESSO KERNEL viene **chiamata `free_kpages()`** per liberare le pag fisiche. Quando termina un PROCESSO UTENTE viene chiamata **`as_destroy()`** per liberare le pag dell'address space tramite `freepages()`.

CREAZIONE DI UN PROCESSO UTENTE

- A) Si crea un nuovo address space con `as_create()`
- B) Si definiscono le regioni di memoria (codice, dati ,stack) con `as_define_region()`
- C) Si allocano le pag in KSEG0 con `as_prepare_load()` che richiama `getppages()`.
`as_prepare_load` usa memoria non liberabile; per implementare `ram_freemem` bisogna modificare `ram_stealmem`. `As_prepare_load` alloca pag fisiche (lato user) con `alloc_kpages()` che usa `ram_stealmem` che non permette di recuperare memoria. Per farlo dobbiamo aggiungere delle strutture dati.
- D) `Getppages()` cerca pag fisiche libere consecutive usando `getfreeppages()`
- E) Se non le trova usa `ram_stealmem()` che fa avanzare il puntatore della RAM.

CREAZIONE PROCESSO KERNEL

Chiama direttamente `alloc_kpages()` che usa `getppages()` e se necessario `ram_stealmem()`

Mentre i processi utente utilizzano address space + mapping con TLB e per questo devono definire le loro zone di memoria, allo stesso tempo i processi kernel allocano direttamente pag fisiche in KSEG0 senza usare meccanismo di traduzione degli indirizzi.

`isTableActive()` è una funzione di supporto che serve per verificare se le tabelle sono state inizializzate o meno:

```

static int
isTableActive(void)
{
    int active;

    spinlock_acquire(&freemem_lock);

    active = allocTableActive;

    spinlock_release(&freemem_lock);

    return active;
}

```

Primitive di Sincronizzazione

```

do {
    entry section
    critical section
    exit section
    remainder section
} while (true);

```

SEZIONE CRITICA = parte di codice che accede ad una risorsa condivisa (variabili globali...). Per questo non può essere eseguita contemporaneamente da + thread/processi, altrimenti abbiamo una RACE CONDITION e ci troviamo in uno stato inconsistente e otteniamo un risultato errato. Dobbiamo avere:

- MUTUAL EXCLUSION: 1 solo thread può entrare nella sezione critica
- PROGRESS : se nessuno è nella sezione critica e un thread deve entrare, non deve aspettare per sempre, ma qualcuno deve decidere chi entra.
- BOUNDED WAITING: per evitare STARVATION → garantisce che un thread che sta aspettando di entrare nella sezione critica non venga continuamente superato da altri, ma prima o poi ottenga il suo turno.

Algoritmi per gestione della critical section dipendono da tipo di kernel:

- SINGLE CORE NON PREEMPTIVE: esiste una sola CPU, quando un thread entra nel kernel non può essere interrotto da altri. Continua ad eseguire finché non termina la sys call, si blocca o fa thread_yield(). Nel kernel c'è solo 1 thread alla volta, quindi no RACE CONDITION e non servono primitive di sincronizzazione.
- PREEMPTIVE SINGLE CORE: 1 sola CPU, ma il SO può interrompere un thread mentre esegue codice del kernel. 2 thread potrebbero eseguire lo stesso codice di kernel (es. Counter ++) in momenti molto ravvicinati → race condition → servono primitive di sincronizzazione.
- MULTI CORE: più CPU in parallelo → servono primitive di sincronizzazione.

ALGORITMO DI PETERSON

Soluzione sw per il problema della mutua esclusione, per 2 soli thread. Utilizza 2 variabili condivise e soddisfa le 3 proprietà citate pri,a:

- bool flag[2] → indica intenzione di ogni thread di entrare nella sezione critica. Es: flag[0] = true => thread 0 desidera entrare nella sezione critica.
- int turn → turno del processo per entrare nella sezione critica

Quando un thread desidera entrare per prima cosa setta flag = true, per dichiarare la sua intenzione di entrare. Poi assegna alla variabile turn l'identificativo dell'altro thread. Così nel caso in cui vogliano entrare entrambi contemporaneamente, è disposto a lasciare la precedenza

```
while (true){
    flag[i] = true;
    turn = j;
    while (flag[j] && turn == j)
        ;

    /* critical section */

    flag[i] = false;

    /* remainder section */
}
```

all'altro. }

Finché le condizioni sono vere attende, altrimenti il ciclo termina ed entra nella sezione critica. Dentro setta flag = false, così lascia all'altro thread la possibilità di entrare. Così non si trovano mai 2 thread contemporaneamente nella stessa sezione critica (mutua esclusione). LIMITE: algoritmo prevede che le istruzioni vengano eseguite esattamente nell'ordine scritto dal programmatore, ma nei sistemi moderni c'è l'instruction reordering...not ok. Quindi le soluzioni sw non bastano, servono primitive hardware atomiche, cioè eseguite come un'unica operazione indivisibile.

3 SOLUZIONI HW:

- Memory Barrier → istruzione che obbliga il processore a completare tutte le operazioni di memoria precedenti prima di proseguire con le successive. Può considerare come modello di memoria: Strongly Ordered (ogni modifica effettuata da un processore diventa immediatamente visibile a tutti gli altri processori), oppure Weakly Ordered (le modifiche possono non essere immediatamente visibili agli altri, quindi dobbiamo usare memory barrier).

Per garantire ordine di visibilità delle operazioni in memoria:

```
x = 100;
memory_barrier();
flag = true;
```

- Hw Atomic instructions: istruzioni eseguite come un'unica operazione indivisibile. Sono :
 - Test and set:

```

while (test_and_set(&lock))
    ;

critical_section();

lock = false;

```

Si fa test su una condizione, se true si setta il valore.

Internamente fa: old = lock; lock = true; return old.

■ Compare and swap:

```

while (compare_and_swap(&lock, 0, 1) != 0)
    ;

critical_section();

lock = 0;

```

Confronto il valore di lock se = 0, setto a 1, altrimenti no .

- Atomic Variables: fornire operazioni già atomiche → atomic_inc(&counter);

Una soluzione moderna è l'utilizzo di un **LOCK**: prima di accedere alla risorsa condivisa il thread deve acquisire un lock. Se libero allora entra, altrimenti aspetta. Terminato il lock, lo rilascia: release(lock). Realizza automaticamente la mutua esclusione. Quando un thread cerca di acquisire (acquire (lock)) un lock già occupato può succedere: Busy Waiting: thread controlla continuamente se il lock si libera, non si sospende e consuma CPU; SLEEP: sospendo completamente il thread che viene inserito in una coda di attesa e la CPU viene assegnata ad un altro thread, quando il lock viene liberato, il thread si risveglia e riprende l'esecuzione.

Lo SPINLOCK usa internamente le istruzioni atomiche e internamente fa qualcosa tipo test_and_set / compare_and_swap.

```

spinlock_acquire(&lk);

/* critical section */

spinlock_release(&lk);

```

MUTEX LOCK

Struttura sw che garantisce che un solo thread alla volta possa entrare nella sezione critica. Il programmatore non si preoccupa di come il lock viene implementato internamente.

```

lock_acquire(lock);

/* critical section */

lock_release(lock);

```

Acquire() e release() devono essere atomiche! Devono usare istruzioni atomiche come test-and-set o compare-and-swap

SCHEMA RICORRENTE:

```

while (true) {

    acquire(lock);

    /* critical section */

    release(lock);

    /* remainder section */

}

```

Uno degli svantaggi è che richiede busy waiting e quindi consuma CPU. Questo tipo di lock è lo SPINLOCK (lock in cui il thread, se non riesce ad acquisire il lock, continua a ciclare). Convieniene solo quando la sezione critica è piccola e l'attesa dura poco.

L'implementazione di SPINLOCK in OS161:

```

struct spinlock {

    int lock;

    struct thread * owner; //owner attuale del thread

}

```

SPINLOCK_ACQUIRE

(kern/thread/spinlock.c)

```

spinlock_acquire(struct spinlock *splk) {
    ...
    while (1) {
        /* Do test-test-and-set, that is, read first before
           doing test-and-set, to reduce bus contention.
           Test-and-set is a machine-level atomic operation
           */
        if (spinlock_data_get(&splk->splk_lock) != 0) {
            continue;
        }
        if (spinlock_data_testandset(&splk->splk_lock) != 0) {
            continue;
        }
        break;
    }
    ...
}

```

Come parametro abbiamo una struct: puntatore allo spinlock da acquisire

Poi inizia il busy waiting (while (1)). Dopo di che legge il valore del lock e se restituisce un valore != 0 vuol dire che il lock è già occupato e quindi fa continue e ricomincia il ciclo. Se invece il lock sembra libero, prova davvero ad acquisirlo con un'operazione atomica testandset (old = splk_lock; splk_lock = 1; return old). Può restituire 0 (lock era realmente libero e quindi lo ha acquistato e termina), 1 (mentre stava leggendo lock, un altro thread lo ha acquistato prima, quindi si ricomincia da capo).

SPINLOCK_RELEASE

```
void
spinlock_release(struct spinlock *splk)
{
    /* this must work before curcpu initialization */
    if (CURCPU_EXISTS()) {
        KASSERT(splk->splk_holder == curcpu->c_self);
        KASSERT(curcpu->c_spinlocks > 0);
        curcpu->c_spinlocks--;
        HUNGMAN_RELEASE(&curcpu->c_hangman, &splk->splk_hangman);
    }

    splk->splk_holder = NULL;
    membar_any_store();
    spinlock_data_set(&splk->splk_lock, 0);
    spllower(IPL_HIGH, IPL_NONE);
}
```

Come parametro riceve il puntatore allo spinlock che deve essere rilasciato. Non restituisce nulla. Se esiste curcpu allora fa alcuni controlli: la Cpu che sta rilasciando lo spinlock deve essere effettivamente quella che lo possiede. Quindi l'holder dello spinlock è la CPU, non il thread. Controlla che la CPU risulti effettivamente proprietaria di almeno uno spinlock. Subito dopo decrementa il numero di spinlock posseduti dalla CPU. HUNGMAN_RELEASE fa parte dell'infrastruttura di debug (per individuare problemi tipo deadlock). Poi viene rimosso l'owner (lo spinlock non è + associato alla CPU corrente), membar_any_store() è una memory barrier per garantire che tutte le scritture effettuate dentro la sezione critica siano completate prima di liberare il lock. **Spinlock_data_set()** è la vera operazione che libera lo spinlock (0). Da questo momento un'altra CPU che sta eseguendo spinlock_acquire potrà vedere il lock libero e acquisirlo. Con spllower() abbassiamo livello interrupt (in modo tale che la CPU non venga interrotta mentre possiede lo spinlock). Spinlock_release() non sveglia nessun thread, perché chi aspetta uno spinlock è in busy waiting. Spinlock_release() NON ha WAITING quindi non è gestita con busy waiting, a differenza di spinlock_acquire.

IN MULTICORE (almeno 2 CPU) NON abbiamo disabling interrupt per ottenere mutua esclusione!!!.

Per spinlock_data_get e spinlock_data_testandset NON servono i SEMAFORI, essendo delle primitive di + basso livello. Al loro interno NON chiamano spinlock_acquire/spinlock_release. Sono funzioni utilizzate per realizzare spinlock_acquire. La funzione spinlock_release non chiama la spinlock_data_testandset ma la spinlock_set, perché non deve gestire una mutua esclusione. The spinlock_acquire function is implemented using the spinlock_data_testandset function, which guarantees atomicity for reading and writing a flag.

SEMAFORI

```

struct semaphore *s;

s = sem_create("MySem1", 1);

P(s);          // entra nella sezione critica

/* Critical Section */

V(s);          // esce dalla sezione critica

```

E' una primitiva di sincronizzazione costituita da : int count (un contatore interno). Può essere modificato solo tramite 2 operazioni atomiche: P() = wait() e V() = signal().

Con 1 intendiamo che una sola risorsa è disponibile, nel nostro caso l'accesso alla sezione critica.

P() = wait() : se il contatore > 0 lo decrementa e il thread prosegue, se count = 0 il thread deve attendere. Abbiamo busy waiting infatti per eliminarla useremo wchan_sleep. Se il contatore è > 0 entra e count --

V() = signal() : count ++ e rende disponibile una nuova risorsa, svegliando i thread in attesa. Count ++ se thread in attesa allora quel thread può essere risvegliato.

```

wait(S)
{
    while (S <= 0)
        ;

    S--;
}

signal(S)
{
    S++;
}

```

Il semaforo può essere:

- Counting Semaphore: il contatore può assumere qualunque valore intero non negativo.
- Binary Semaphore = assume solo 2 valori 0 (risorsa occupata) e 1 (risorsa libera). Simile a mutex.

2 thread non devono poter eseguire contemporaneamente wait() e signal() sullo stesso semaforo. Una buona soluzione è smettere di continuare a controllare il semaforo (busy waiting) e il thread si mette a dormire (block) e verrà risvegliato (wakeup) non appena il semaforo torna disponibile.

LOCK : kern/include/sync.h

```

//
struct lock {
    char *lk_name;
    HANGMAN_LOCKABLE(lk_hangman); /* Deadlock detector hook. */
    // add what you need here
    // (don't forget to mark things volatile as needed)
    #if OPT_SYNCH
    #if USE_SEMAPHORE_FOR_LOCK
        struct semaphore *lk_sem;
    #else
        struct wchan *lk_wchan;
    #endif
    struct spinlock lk_lock;
    volatile struct thread *lk_owner;
    #endif
};

```

HANGMAN_LOCKABLE viene usato per debug e individuare deadlock tra lock. Poi abbiamo il nome del lock (lk_name). In questo caso costruiamo il lock con un semaforo binario (quando il lock viene creato → sem_create(name, 1), nasce con count = 1, ossia lock libero). Quando un thread esegue lock_acquire(lock) internamente viene chiamato P(lock->lk_sem), quando invece esegue lock_release(lock) viene chiamato V(lock->lk_sem). Se invece il lock non utilizzasse più un semaforo, userebbe un **wait channel** ossia una coda di attesa dei thread. Poi utilizziamo un lock **(lk_lock)** per proteggere i dati interni del lock, ossia **lk_owner**, per evitare una race condition. **Lk_owner contiene il thread che possiede il lock, serve a verificare che il lock venga rilasciato dallo stesso thread che lo ha acquisito. Volatile serve per impedire ottimizzazioni del compilatore e andare sempre a rileggere il valore in memoria.**

Quindi lk_sem implementa il comportamento del lock (attesa e risveglio con P() e V()), invece lk_lock protegge le variabili interne a lock come lk_owner durante aggiornamenti per evitare race condition. Lk_owner è un puntatore ad una struct thread.

LOCK_CREATE (kern/thread/sync.c)

```

struct lock *
lock_create(const char *name)
{
    struct lock *lock;

    lock = kmalloc(sizeof(*lock));
    if (lock == NULL) {
        return NULL;
    }

    lock->lk_name = kstrdup(name);
    if (lock->lk_name == NULL) {
        kfree(lock);
        return NULL;
    }

    HANGMAN_LOCKABLEINIT(&lock->lk_hangman, lock->lk_name);
}

```

```

// add stuff here as needed
#if OPT_SYNCH
#if USE_SEMAPHORE_FOR_LOCK
    lock->lk_sem = sem_create(lock->lk_name, 1);
    if (lock->lk_sem == NULL) {
        kfree(lock->lk_name);
        kfree(lock);
        return NULL;
    }
#else
    lock->lk_wchan = wchan_create(lock->lk_name);
    if (lock->lk_wchan == NULL) {
        kfree(lock->lk_name);
        kfree(lock);
        return NULL;
    }
#endif
    lock->lk_owner = NULL;
    spinlock_init(&lock->lk_lock);
    #endif
    return lock;
}

```

PARAMETRO = nome del lock

VALORE DI RITORNO = puntatore a nuovo lock se la creazione ha successo (lk_name, lk-sem, lk_owner, lk_lock). NULL se si verifica un errore.

Per prima cosa si alloca la memoria necessaria per contenere la struct del lock e si controlla se c'è memoria disponibile. Con `kstrdup(name)` si fa una copia della stringa del nome e si vede se è fallita o meno. Si inizializza la struttura usata dal debugger Hangman. Si crea il semaforo binario inizializzato a 1 (`lock->lk_sem = sem_create`). Nel caso in cui `sem_create()` fallisca si libera tutto quello che era stato allocato per evitare memory leak. All'inizio nessun thread possiede il lock quindi `lk_owner = NULL`, poi dopo `lock_acquire()` si aggiornerà l'owner. Si inizializza lo spinlock interno per proteggere `lk_owner`.

L'altra alternativa, al posto dei semafori, è implementare il lock con il wait channel, ma in entrambe le implementazioni si deve sapere chi possiede il lock e devono proteggere i dati interni.

LOCK_ACQUIRE

```
lock_acquire(struct lock *lock)
/* Call this (atomically) before waiting for a lock */
//HANGMAN_WAIT(&curthread->t_hangman, &lock->lk_hangman);

// Write this
#if OPT_SYNC
    KASSERT(lock != NULL);

    KASSERT(curthread->t_in_interrupt==false);
#endif
#if USE_SEMAPHORE_FOR_LOCK
    P(lock->lk_sem);
    spinlock_acquire(&lock->lk_lock);
#else
    spinlock_acquire(&lock->lk_lock);
    while(lock->lk_owner != NULL) {
        wchan_sleep(lock->lk_wchan, &lock->lk_lock);
    }
#endif
    KASSERT(lock->lk_owner == NULL);
    lock->lk_owner = curthread;
    spinlock_release(&lock->lk_lock);
#endif

(void)lock; // suppress warning until code gets written

/* Call this (atomically) once the lock is acquired */
//HANGMAN_ACQUIRE(&curthread->t_hangman, &lock->lk_hangman);
}
```

Ritorna void e prende come parametro il puntatore al lock che il thread corrente vuole acquisire. Per prima cosa verifichiamo che il puntatore al loc sia valido con un `KASSERT` e che `KASSERT(curthread -> t_in_interrupt == false)` e quindi che se qualcuno tenta di fare `lock_acquire()` durante un interrupt, il kernel segnala subito un errore, perché `lock_acquire()` non può essere usata in interrupt context perché può bloccare il thread e durante un interrupt non si può andare a dormire. Nella versione con il SEMAFORO, `P(lock->lk_sem)` acquisisce il semaforo binario, se il semaforo vale 1 il thread entra immediatamente, altrimenti viene sospeso finché non viene rilasciato. Una volta ottenuto il lock, acquisisce lo spinlock per aggiornare in mutua esclusione i dati interni del lock (`lk_owner`). Nella versione con **wchan** : thread acquisisce lo spinlock e controlla se il lock è già occupato . Se `lk_owner != NULL` (esiste già un proprietario), allora si addormenta sul wait channel. Quando viene svegliato, `wchan_sleep()`

riacquista automaticamente lo spinlock e il controllo viene ripetuto dal while. Infine viene registrato come owner il thread corrente. Da questo momento solo lui può eseguire lock_release(). Infine viene rilasciato lo spinlock.

LOCK_DESTROY()

```
void
lock_destroy(struct lock *lock)
{
    KASSERT(lock != NULL);

    // add stuff here as needed
    #if OPT_SYNCH
        spinlock_cleanup(&lock->lk_lock);
    #if USE_SEMAPHORE_FOR_LOCK
        sem_destroy(lock->lk_sem);
    #else
        wchan_destroy(lock->lk_wchan);
    #endif
    #endif

    kfree(lock->lk_name);
    kfree(lock);
}
```

Come parametro riceve il puntatore al lock che deve essere distrutto e non ritorna niente. Per prima cosa dobbiamo controllare : che il **lock esista** (!= NULL, non ha senso distruggere un lock inesistente) e **che non è posseduto da nessun thread** (lk_owner == NULL). Per prima cosa puliamo (spinlock_cleanup()) lo spinlock interno che era stato inizializzato in lock_create() con spinlock_init(). **Non facciamo kfree() dello spinlock**, perché lk_lock è direttamente un campo della struct lock, non una struttura allocata separatamente. Poi distruggiamo la struttura usata per implementare l'attesa, o un semaforo (sem_destroy) oppure il wait channel (wchan_destroy). Infine liberiamo la memoria della stringa e della struct stessa.

LOCK_RELEASE

```
void
lock_release(struct lock *lock)
{
    /* Call this (atomically) when the lock is released */
    //HANGMAN_RELEASE(&curthread->t_hangman, &lock->lk_hangman);

    // Write this
    #if OPT_SYNCH
        KASSERT(lock != NULL);
        KASSERT(lock_do_i_hold(lock));
        spinlock_acquire(&lock->lk_lock);
        lock->lk_owner=NULL;

    #if USE_SEMAPHORE_FOR_LOCK
        V(lock->lk_sem);
    #else
        wchan_wakeone(lock->lk_wchan, &lock->lk_lock);
    #endif
        spinlock_release(&lock->lk_lock);
    #endif

    (void)lock; // suppress warning until code gets written
}
```

Riceve come parametro il lock da rilasciare e non restituisce nulla. Per prima cosa controlliamo che il lock esista e che il THREAD CORRENTE SIA DAVVERO L'OWNER DEL LOCK (lock_do_i_hold(lock)). **A differenza dei semafori (dove qualsiasi thread può eseguire P(), V()), il lock ha OWNERSHIP: solo il thread che ha fatto lock_acquire() può fare lock_release().**

Infatti si usa per verificare ownership `lock_do_i_hold()`. Si acquisisce lo spinlock per acquisire accesso su `lk_owner`. Si segna il lock come non più posseduto e nella versione con SEMAFORO: `V(lock->lk_sem)→` si incrementa il semaforo binario, quindi il lock torna disponibile e un eventuale thread bloccato in `P()` viene risvegliato. Nella versione con WCHAN: `wchan_wakeone()` si risveglia un solo thread in attesa su wait channel. Infine si rilascia lo spinlock interno.

`LOCK_DO_I_HOLD` per verificare ownership

```
bool
lock_do_i_hold(struct lock *lock)
{
    // Write this
    #if OPT_SYNC
        bool res;

        spinlock_acquire(&lock->lk_lock);
        res = lock->lk_owner == curthread;
        spinlock_release(&lock->lk_lock);
        return res;
    #endif

    (void)lock; // suppress warning until code gets written

    return true; // dummy until code gets written
}
```

Riceve il lock da verificare. Si acquisisce lo spinlock per accedere in mutua esclusione al `lk_owner`, essendo una variabile condivisa tra + thread. Si confronta il proprietario del lock con il thread corrente, se coincidono allora true, altrimenti false. Però prima si rilascia lo spinlock.

CONDITION VARIABLE

La condvar non protegge i dati, serve solo a far dormire e risvegliare i thread quando una condizione cambia. La protezione dei dati avviene con un lock.

```
struct cv {
    char *cv_name;
    // add what you need here
    // (don't forget to mark things volatile as needed)
    #if OPT_SYNC
        struct wchan *cv_wchan;
        struct spinlock cv_lock;
    #endif
};
```

La struttura contiene: nome della condvar, `cv_wchan` è il wait channel, ossia la coda dei thread in attesa e lo spinlock interno della condvar che protegge il wait channel durante operazioni di `wchan_sleep()`, `wchan_wakeone()`, `wchan_wakeall()`. Quando un thread esegue `cv_wait(cv, lock)` non rimane in busy waiting ma viene inserito dentro `cv->cv_wchan` e viene sospeso. Più avanti un altro thread farà `cv_signal(cv, lock)` oppure `cv_broadcast(cv, lock)` che risveglierà uno solo o tutti i thread presenti in coda. Le funzioni `wchan_sleep()`, `wchan_wakeone()`, `wchan_wakeall()` richiedono obbligatoriamente uno spinlock e non un lock semplice, per proteggere accesso concorrente alla coda `wchan`. `Wchan_sleep()` rilascia automaticamente questo spinlock quando il thread si addormenta e lo riacquista quando si risveglia.

Quindi le condvar si implementano con: **wait channel + spinlock**.

CV_CREATE

```
struct cv *
cv_create(const char *name)
{
    struct cv *cv;

    cv = kmalloc(sizeof(*cv));
    if (cv == NULL) {
        return NULL;
    }

    cv->cv_name = kstrdup(name);
    if (cv->cv_name==NULL) {
        kfree(cv);
        return NULL;
    }

    // add stuff here as needed
    #if OPT_SYNC
        cv->cv_wchan = wchan_create(cv->cv_name);
        if (cv->cv_wchan == NULL) {
            kfree(cv->cv_name);
            kfree(cv);
            return NULL;
        }
        spinlock_init(&cv->cv_lock);
    #endif

    return cv;
}
```

Per prima cosa alloca la struttura nel kernel (se fallisce ritorna NULL), crea una copia del nome. Poi viene creato il wait channel, la coda sulla quale dormiranno tutti i thread che eseguono `cv_wait()`, con `wchan_create()`. Se la creazione fallisce vengono liberate tutte le risorse allocate fino a quel momento. Infine viene inizializzato lo spinlock associato al wait channel per proteggere la coda di thread (evitando race condition nell'accesso concorrente alla coda).

CV_DESTROY

```
void
cv_destroy(struct cv *cv)
{
    KASSERT(cv != NULL);

    // add stuff here as needed
    #if OPT_SYNC
        spinlock_cleanup(&cv->cv_lock);
        wchan_destroy(cv->cv_wchan);
    #endif

    kfree(cv->cv_name);
    kfree(cv);
}
```

Distrugge completamente una condvar, liberando tutte le risorse allocate in `cv_create()`. Verifico che `cv` non sia NULL, altrimenti starei tentando di distruggere una `cv` inesistente. Distruggo lo spinlock associato alla `cv` con `spinlock_cleanup()` e distruggo il wait channel. Libero il nome e la struct `cv`.

CV_WAIT

```

void
cv_wait(struct cv *cv, struct lock *lock)
{
    // Write this
    #if OPT_SYNCH
        KASSERT(lock != NULL);
        KASSERT(cv != NULL);
        KASSERT(lock_do_i_hold(lock));

        spinlock_acquire(&cv->cv_lock);

        lock_release(lock);
        wchan_sleep(cv->cv_wchan, &cv->cv_lock);
        spinlock_release(&cv->cv_lock);

        lock_acquire(lock);
    #endif
    (void)cv; // suppress warning until code gets written
    (void)lock; // suppress warning until code gets written
}

```

Verifico che il lock e la cv esistano, verifico che il thread corrente possieda il lock. Il 3 KASSERT è fondamentale perché una condvar funziona sempre insieme ad un lock!!! Non si può chiamare cv_wait() senza possedere il lock. Infatti di solito:

```

lock_acquire(lock);

while (!condizione)
    cv_wait(cv, lock);

lock_release(lock);

```

Si acquisisce lo spinlock che protegge la coda dei thread in attesa, si rilascia il lock così che il thread rilascia il mutex e mentre lui aspetta gli altri thread possono entrare nella sezione critica e modificare la condizione. Poi abbiamo **wchan_sleep()** che permette al thread di entrare nella coda del wchan, si addormenta e rilascia automaticamente lo spinlock durante il context switch. Quando si risveglia riprende automaticamente lo spinlock. Rilascia lo spinlock della CP ora che il thread è stato risvegliato e riprende il lock. Senza spinlock si sarebbe potuto perdere il signal e avremmo avuto lost wakeup, un thread si addormentava per sempre. Viene evitato con questa sequenza, così il segnale non viene perso: spinlock_acquire(cv_lock); lock_release(lock); wchan_sleep(...);. Tra il rilascio del lock e l'ingresso in coda nessuno infatti può fare cv_signal() perché dovrebbero acquisire cv_lock.

Realizza un'attesa senza busy waiting.

Wchan è una primitiva interna del kernel, serve solo a mettere un thread a dormire e risvegliarlo. Riceve uno spinlock. Invece le condvar sono delle primitive di sincronizzazione costruite sopra il wchan.

CV_SIGNAL

Riceve condvar e lock associato. **Cosa importante: chi fa cv_signal() deve possedere il lock, in modo tale che la condizione condivisa venga modificata e segnalata in mutua esclusione (KASSERT(lock_do_i_hold(lock))).** Poi si acquisisce lo spinlock interno della CV per

accedere al wait channel e con `wchan_wakeone()` si risveglia 1 SOLO THREAD di quelli che fanno dormendo sulla CV. Infine si rilascia lo spinlock.

```
void
cv_signal(struct cv *cv, struct lock *lock)
{
    #if OPT_SYNC
        KASSERT(lock != NULL);
        KASSERT(cv != NULL);
        KASSERT(lock_do_i_hold(lock));

        spinlock_acquire(&cv->cv_lock);
        wchan_wakeone(cv->cv_wchan, &cv->cv_lock);
        spinlock_release(&cv->cv_lock);
    #endif

    (void)cv;
    (void)lock;
}
```

`Cv_broadcast()` è praticamente identica, solo che al posto di `wchan_wakeone()` ha **`wchan_wakeall()`**. Risveglia tutti i thread

Per i semafori: entrambi ricevono come parametro la struct semaphore

P():

V()

```
spinlock_acquire(&sem->sem_lock);

while (sem->sem_count == 0) {
    wchan_sleep(sem->sem_wchan, &sem->sem_lock);
}

sem->sem_count--;

spinlock_release(&sem->sem_lock);
```

```
spinlock_acquire(&sem->sem_lock);

sem->sem_count++;

wchan_wakeone(sem->sem_wchan, &sem->sem_lock);

spinlock_release(&sem->sem_lock);
```

In P(): `sem_lock()` protegge il contatore interno del semaforo, con `sem_count==0` il thread non fa busy waiting ma va a dormire con `wchan_sleep()`. Quando si risveglia ricontrolla il while e se `count > 0` allora decrementa il contatore. Il while è fondamentale perché quando il thread si sveglia non è garantito che la risorsa sia ancora disponibile, un altro thread potrebbe averla presa prima.

In V(): prende lo spinlock, incrementa il contatore, risveglia un thread e rilascia lo spinlock.

Questa funzione (`wchan_sleep()`) invece richiede di avere già lo spinlock:

```
void
wchan_sleep(struct wchan *wc, struct spinlock *lk)
{
    KASSERT(!curthread->t_in_interrupt);
    KASSERT(spinlock_do_i_hold(lk));
    KASSERT(curcpu->c_spinlocks == 1);

    thread_switch(S_SLEEP, wc, lk);

    spinlock_acquire(lk);
}
```

Con `thread_switch` mette il thread in stato `S_SLEEP` e durante il context switch rilascia lo spinlock. Così evitiamo il deadlock: TA possiede spinlock, si mette a dormire senza rilasciarlo, TB non può acquisirlo e non può cambiare condizione, quindi TA non viene mai risvegliato. Quando il thread viene risvegliato e ritorna da `thread_switch`, riacquisisce lo spinlock prima di ritornare da `wchan_sleep()`.

Con `wchan_wakeone()` :

```
target = threadlist_remhead(&wc->wc_threads);

if (target == NULL) {
    return;
}

thread_make_runnable(target, false);
```

Prende il primo thread dalla lista del wait channel, se non c'è nessuno non fa nulla, altrimenti lo rende di nuovo runnable, ma non parte immediatamente !!!.

Terminazione di un processo

PROCESSO ZOMBIE = processo che ha terminato l'esecuzione, quindi il suo thread non sta più eseguendo codice user, ma la sua struct `proc` e alcuni metadata devono ancora rimanere disponibili perché un altro processo deve poterne recuperare lo **stato di uscita**. Dopo il `thread_exit()` dell'ultimo thread del processo, il processo rimane zombie finché qualcuno non esegue `wait/waitpid` e ne recupera l'exit status. Quindi quando un processo termina, non possiamo distruggere subito la struct `proc`, perché altrimenti perderemmo il PID, exit status, e i metadata necessari a `waitpid`. Dobbiamo invece salvare lo exit status, segnalare la terminazione, terminare il thread, la struct `proc` rimane zombie, il padre esegue `wait` e infine distrugge la struct `proc`. `sys__exit()` segnala la terminazione, mentre `proc_wait()` recupera lo stato e distrugge la struct `proc` dopo il risveglio. Nell'implementazione iniziale di `sys__exit()` infatti la struct `proc` del processo viene persa quando l'address space viene rilasciato.

Si parte da una funzione interna al kernel: **`proc_wait()`**

```
int proc_wait(struct proc *proc) {
    #if OPT_WAITPID
        int return_status;

        P(proc->p_sem);

        return_status = proc->p_status;
        proc_destroy(proc);
        return return_status;
    #else
        void(proc);
        return 0;
    #endif
}
```

Aspetta che il processo figlio termini e restituisce il suo codice di uscita.

Riceve direttamente il puntatore al processo che vogliamo aspettare, non serve il PID.

E' importante che prima si faccia P() e poi proc_destroy(), altrimenti rischieremmo che la struct proc venga liberata prima e si cercherebbe di accedere a memoria già liberata. Questa funzione sa già quale processo aspettare perché riceve la struct proc.

La struct proc deve essere modificata in questo modo:

```
struct proc {
    char *p_name;           /* Name of this process */
    struct spinlock p_lock; /* Lock for this structure */
    unsigned p_numthreads;  /* Number of threads in this process */

    /* VM */
    struct addressspace *p_addressspace; /* virtual address space */

    /* VFS */
    struct vnode *p_cwd;      /* current working directory */

    /* add more material 0here as needed */
    #if OPT_WAITPID
        int p_status; /* status for exit() */
        pid_t p_pid; /* process pid*/
        struct semaphore *p_sem;
    #endif
};
```

Quindi dobbiamo usare un semaforo privato per la sincronizzazione (p_sem), aggiungere lo stato di terminazione (p_status) e l'identificatore del processo (p_pid). Al posto del semaforo si potevano usare anche le condvar in questo modo:

```
lock_acquire(proc->p_lock);
```

```
cv_wait(proc->p_cv);
```

```
lock_release(proc->p_lock);
```

Quando eseguiamo ad es p testbin/palin, il kernel (padre) crea il processo figlio invocando common_prog() che contiene proc_create_runprogram() che crea la struct proc. Poi con thread_fork() il kernel crea il thread figlio e questo inizia ad eseguire. Abbiamo 2 thread. Il padre non vuole terminare subito, ma vuole aspettare la terminazione del figlio, quindi chiama proc_wait(proc), che dentro ha un semaforo con count = 0, fa P(proc->p_sem) e quindi il padre non può entrare e si mette a dormire (no busy waiting). Nel frattempo il figlio esegue cmd_progtread(), runprogram(), il programma user, il main() e poi chiama exit(). Con questa si entra nel kernel, non si distrugge il processo, ma si chiama la sys__exit e quindi viene salvato il codice di uscita, si segnala al padre per risvegliarlo, con V(p->p_sem) così il count = 1, si termina il thread con thread_exit() così che il figlio muore. Il padre si risveglia (appena il figlio fa V(p_sem)), il padre legge lo stato di uscita del figlio e distrugge il processo con proc_destroy(proc).

Il pid nella struct proc mi serve perché quando faremo waitpid(pid...) il kernel usa questo identificatore per capire quale processo il chiamante vuole aspettare. Il semaforo è privato del processo. E' importante salvare anche lo stato, perché con thread_exit() il thread muore e perderemmo tutto. Ogni processo ha il proprio semaforo.

SYS_EXIT

```
void sys_exit(int status) {    //parametro: stato di terminazione
    #if OPT_WAITPID
        struct proc *p = curproc;
        p->p_status = status & 0xff;
        proc_remthread(curthread);
        V(p->p_sem);
    #else
        struct addrspace *as = proc_getas();
        as_destroy(as);
    #endif
    thread_exit();

    panic("Panic! \n");
    (void) status;
}
```

Con le condvar, al posto di V() avrei avuto:

```
lock_acquire(p->p_lock);
```

```
cv_signal(p->p_cv);
```

```
lock_release(p->p_lock);
```

Curproc è il processo corrente, quello che esegue la exit(). P è il puntatore alla struct proc. Si salvano gli 8 bit meno significativi del codice di uscita passato da exit(). Con proc_remthread() si stacca il thread dal processo. Questa deve essere fatta prima di V() altrimenti rischieremmo una race condition: il padre si sveglia e chiama proc_destroy() distruggendo la struct proc quando il thread figlio risulta ancora collegato. Con V() sveglia il padre perché incrementa il contatore. **Termina definitivamente il thread processo figlio con thread_exit() ma NON viene distrutta la struct proc!!!! Rimane in memoria come processo zombie.** Proc_wait() fa proc_destroy() e distrugge la struct proc. Il panic non dovrebbe essere mai eseguito.

Thread_exit() va modificato aggiungendo questo:

```
if (curthread->t_proc != NULL) {
    proc_remthread(curthread);
}
```

per gestire i thread ancora collegati al processo, nel caso in cui un thread termini.

Ora introduciamo la **TABELLA GLOBALE DEI PROCESSI: vettore di puntatori il cui indice è il PID**

perché waitpid riceve un PID, non un puntatore ad una struct proc come proc_wait().

```
#define MAX_PROC 100
static struct _processTable {
    int active;
    struct proc * proc[MAX_PROC+1];
    int last_i;
    struct spinlock lk;
} processTable;

#endif
```

Active dice se la tabella è stata già inizializzata (0 all'avvio, 1 dopo proc_bootstrap()). Proc è un vettore che associa direttamente un PID alla struct proc (es proc[1] = processo con PID1...).

lk è uno spinlock che protegge la tabella da accessi concorrenti. Last_i memorizza l'ultimo PID assegnato, in modo da ripartire dal prossimo PID libero. Serve questa tabella perché con waitpid() il programma utente passa solo un intero e dato un PID e basta il kernel non saprebbe quale struct proc recuperare.

PROC_SEARCH_PID: traduce un PID nella corrispondente struct proc *. Dato un PID restituisce il processo associato. Accesso diretto tramite indice.

```
struct proc * proc_search_pid(pid_t pid) {
    #if OPT_WAITPID
        struct proc *p;
        p = processTable.proc[pid];
        return p;
    #else
        (void) pid;
        return NULL;
    #endif
}
```

PROC_INIT_WAITPID

Quando viene creato un nuovo processo con **proc_create()** dobbiamo assegnargli un PID, inserirlo nella processTable, inizializzare il codice di uscita, creare il semaforo su cui lo aspetta il padre. Per questo proc_create() va modificata, aggiungendo la chiamata a proc_init_waitpid() che prende in ingresso il **puntatore alla struct proc** del processo e il **nome del semaforo**. Oppure se uso le condvar al posto di creare il semaforo faccio:

```
proc->p_cv = cv_create(name, 0);
```

```
proc->p_lock = lock_create(name);
```

```
static void proc_init_waitpid(struct proc *proc, const char *name) {
    #if OPT_WAITPID
        int i;
        spinlock_acquire(&processTable.lk);
        i = processTable.last_i + 1;
        proc->p_pid = 0;
        if(i > MAX_PROC) i=1;
        while(i != processTable.last_i) {
            if(processTable.proc[i] == NULL) {
                processTable.proc[i] = proc;
                processTable.last_i = i;
                proc->p_pid = i;
                break;
            }
            i++;
            if(i > MAX_PROC) i=1;
        }
        spinlock_release(&processTable.lk);
        if(proc->p_pid == 0) {
            panic("too many processes, proc table full\n");
        }
        proc->p_status = 0;
        proc->p_sem = sem_create(name, 0);
    #else
        (void) proc;
        (void) name;
    #endif
}
```

La processTable è condivisa, quindi entriamo in mutua esclusione (se 2 thread creassero contemporaneamente un processo potrebbero assegnare lo stesso PID, quindi dobbiamo usare spinlock!!!).

In i abbiamo ultimo PID assegnato. Se supero MAX_PROC riparto da 1. Cerco il primo PID libero e quando processTable.proc[i] = NULL vuol dire che ho trovato una posizione libera. Inserisco il processo nella tabella, aggiorno ultimo pid con i, salvo il p_pid nella struct proc, esco dal ciclo e rilascio lo spinlock. Inizializzo l'exit status del processo appena nato e creo il semaforo o condvar in cui aspetterà il padre.

PROC_END_WAITPID

```
static void proc_end_waitpid(struct proc *proc) {
    #if OPT_WAITPID
        int i;
        spinlock_acquire(&processTable.lk);
        i = proc->p_pid;
        processTable.proc[i] = NULL;
        spinlock_release(&processTable.lk);
        sem_destroy(proc->p_sem);
    #else
        (void)proc;
    #endif
}
```

Quando eliminiamo un processo con proc_destroy() dobbiamo rimuoverlo dalla process table e distruggere il suo semaforo. Per questo dobbiamo chiamare proc_end_waitpid() al suo interno. Liberiamo il PID, rimuoviamo dalla process table (processTable.proc[i] = NULL), distruggiamo il semaforo o la condvar.

In proc_bootstrap() inizializziamo la processTable:

```
void
proc_bootstrap(void)
{
    kproc = proc_create("[kernel]");
    if (kproc == NULL) {
        panic("proc_create for kproc failed\n");
    }
    spinlock_init(&processTable.lk);
    processTable.active = 1;
}
```

Viene eseguita all'avvio del kernel e prima crea il processo, kproc è la struct proc associata al kernel stesso, poi inizializza lo spinlock che protegge la tabella dei processi e indica che la tabella dei processi è pronta per essere utilizzata : active = 1.

```

pid_t sys_getpid(void) {
    #if OPT_WAITPID
        return curproc -> p_pid;
    #else
        return -1;
    #endif
}

int sys_waitpid(pid_t pid, userptr_t statusp, int options) {
    #if OPT_WAITPID
        struct proc *p = proc_search_pid(pid);
        int s;
        (void)options;
        if (p==NULL){
            return -1;
        }
        s = proc_wait(p);
        if(statusp!=NULL) {
            *(int*) statusp = s;
        }
        return pid;
    #else
        (void)options;
        (void) pid;
        (void)statusp;
        return -1;
    #endif
}

```

Sys_getpid() restituisce semplicemente il p_pid del processo corrente (non di un altro processo). Invece sys_waitpid() è la vera funzione che permette di aspettare un processo conoscendone il PID, lato kernel. Quindi prima crea il processo e quindi chiama proc_search_pid(pid) per ottenere la struct proc del processo creato (puntatore). Se non trova alcun processo associato al PID, waitpid fallisce. Poi con proc_wait() aspetta la terminazione del processo. Infine restituisce lo status di uscita del programma user, statusp p il puntatore passato dal programma user per ricevere lo stato di uscita. Restituisce il PID del processo che è stato ATTESO.

Sys_waitpid() NON ha bisogno di sys_getpid perché il PID lo riceve già come parametro. Sys_getpid è necessaria per implementare la chiamata di sistema getpid, ma viene chiamata dal KERNEL NON da UN PROCESSO UTENTE.